

LEARNING NON-LINEAR DAMPED OSCILLATION USING PINNS

A DISSERTATION

Submitted in the partial fulfilment of the requirement for the award of the degree

of

MASTER OF TECHNOLOGY

in

DATA SCIENCE

Submitted by

ARPIT KUMAR CHAUDHARY

(Enrolment No. - 24566006)

Under the guidance of

Dr. SOHOM RAY

Assistant Professor (Joint)

MEHTA FAMILY SCHOOL OF DATA SCIENCE AND ARTIFICIAL
INTELLIGENCE



MEHTA FAMILY SCHOOL OF DATA SCIENCE AND ARTIFICIAL
INTELLIGENCE

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

ROORKEE – 247667 (INDIA)

MAY 2026

CANDIDATE'S DECLARATION

I declare that this written submission represents my work and ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity. I have not misrepresented or fabricated or falsified any idea / data / fact / source in my submission. I understand that any violation of the above will be cause for disciplinary action by the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date:

Arpit Kumar Chaudhary

Place:

(Enroll. No. - 24566006)

CERTIFICATE

I hereby declare that the work presented in the dissertation entitled “**Learning Non-Linear Damped Oscillation using Physics-Informed Neural Networks**” in the partial fulfilment of requirements for the award of the degree of Master of Technology in Data Science submitted to the Mehta Family School of Data Science and Artificial Intelligence, Indian Institute of Technology, Roorkee, is an authentic record of my work carried out under my supervision. This project does not form the basis for the award of any other degree to the candidate or to anybody else.

Dr. Sohom Ray

Assistant Professor

Department of Earthquake Engineering

Joint Faculty Dept. of

Mehta Family School of Data Science

and Artificial Intelligence

IIT Roorkee

ACKNOWLEDGEMENTS

I would like to express my deep sense of gratitude to my supervisor, **Dr. Sohom Ray**, Associate Professor, Department of Earthquake Engineering and Joint Faculty at Mehta Family School of Data Science and Artificial Intelligence, for their valuable suggestions and guidance, which played a definite role in bringing this report to good shape.

I also extend my sincere thanks to our Head of Department, **Dr. Sanjeev Kumar**, for providing us lab resources, including the NVIDIA RTX A6000 workstation on which all the experiments presented in this thesis were carried out.

I also convey my thanks to all the faculty members of the Mehta Family School of Data Science and Artificial Intelligence, and to my very supportive batch mates, for the many illuminating discussions over the course of two semesters that refined my thinking on physics-informed machine learning and scientific computing. Above all, I thank my family and the Almighty for the firm platform they gave me to complete this work.

Date:

Arpit Kumar Chaudhary

Place:

(Enroll No. - 24566006)

ABSTRACT

This thesis presents a three-phase progressive study of Physics-Informed Neural Networks (PINNs) applied to damped oscillation, beginning with a reading of the foundational IEEE paper of Jiang *et al.* [1] and culminating in a thoroughly-tuned, multi-seed implementation on the cubic Duffing oscillator. Each phase introduces a new layer of complexity and yields a new set of lessons.

Phase 0 surveys the foundational PINN literature, with particular focus on the Jiang *et al.* 2024 IEEE paper that solves linear and forced damped oscillation systems using PINNs.

Phase 1 reproduces the IEEE-paper methodology on a linear damped oscillator $m\ddot{x} + c\dot{x} + kx = F$ and extends it with two baselines: a pure data-driven neural network (black-box) and a Grey-Box PINN that jointly estimates the physical parameters (m, c, k) from sparse noisy measurements. On the in-domain $[0, 2]$ s training window the PINN reached an L2-relative error of 2.2×10^{-3} against the analytical solution. The Grey-Box variant achieved the best $[0, 4]$ s combined error of 0.2395, demonstrating that joint parameter identification can outperform a fixed-parameter PINN when long-time extrapolation matters.

Phase 2.1 generalises the framework to the non-linear regime by introducing quadratic-in-velocity damping, $\ddot{x} + c|\dot{x}|\dot{x} + \omega^2 x = 0$, which models air-drag-like dissipation. A four-layer 160-unit MLP with adaptive initial-condition weighting and Adam-only optimisation at learning rate 2.5×10^{-3} achieved $\text{MSE} = 7.86 \times 10^{-4}$, L2-relative error 8.66×10^{-2} , and maximum absolute error 5.97×10^{-2} against a fourth-order Runge–Kutta reference on $t \in [0, 10]$.

Phase 2.2 re-engineers the framework with three key changes: (i) the cubic-stiffness Duffing equation $\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2 x + \alpha x^3 = 0$ as a more challenging benchmark; (ii) a bounded hard-initial-condition reparametrisation $x_\theta(t) = x_0 + tv_0 + (1 - e^{-t/\tau})^2 \text{NN}_\theta(t)$ that exactly satisfies the IC without polynomial blow-up at long horizons; and (iii) a two-stage Adam-followed-by-L-BFGS optimiser schedule. With a smaller four-layer 64-unit MLP and a 500-step L-BFGS polish, the framework achieved a five-seed mean L2-relative error of $5.67 \times 10^{-4} \pm 8.13 \times 10^{-4}$ on the headline benchmark, with the worst seed at 2.19×10^{-3} – a $130\times$ improvement over the Phase 2.1 number. A grouped-ensemble training scheme implemented through batched matrix multiplication saturates the RTX A6000 GPU at 38 GB of VRAM with $K = 16$ parallel networks, providing per-seed error bars at the cost of a single training run.

The three-phase progression yields three principal findings: (i) the optimiser schedule,

specifically the addition of a 500-step L-BFGS polish on top of Adam, reduces L2 by approximately three orders of magnitude on best-case seeds; (ii) hard-initial-condition reparametrisation matches the best-tuned soft-IC variant in mean error while removing the loss-balancing hyperparameter altogether; and (iii) PINN training quality exhibits substantial seed-to-seed variance that is best characterised by reporting mean $\pm$ std over ≥ 5 seeds rather than single-seed best-of-N numbers.

Keywords: physics-informed neural networks; damped Duffing oscillator; Van der Pol oscillator; hard initial conditions; L-BFGS optimisation; gradient-norm loss balancing; neural tangent kernel; grouped-ensemble training; automatic differentiation; scientific machine learning.

Contents

Abstract	4
List of Symbols and Abbreviations	11
1 Introduction	13
1.1 Background and Motivation	13
1.1.1 Why Beginner-Friendliness Matters Here	14
1.2 Problem Statement	14
1.3 Contributions	15
1.4 Thesis Organisation	15
2 Theoretical Background	17
2.1 The Damped Harmonic Oscillator: A Beginner's Review	17
2.1.1 The Equation and What Each Term Means	17
2.1.2 The Closed-Form Solution (Under-Damped Free Response)	18
2.1.3 Non-Linear Variants	18
2.2 Neural Networks for Beginners	19
2.2.1 The Multi-Layer Perceptron	19
2.2.2 Why Neural Networks Can Approximate Anything (Briefly)	19
2.2.3 Input Normalisation: A Small but Critical Detail	19
2.3 Automatic Differentiation in One Page	20
2.4 Optimisation: Adam and L-BFGS	20
2.4.1 Adam in Three Lines	20
2.4.2 L-BFGS for the Polish	20

2.5	Physics-Informed Neural Networks	21
2.5.1	The Loss Formulation	21
2.5.2	Soft vs Hard Initial-Condition Enforcement	21
2.5.3	Known Failure Modes	21
3	Phase 0 – Literature Foundation	22
3.1	Reading the IEEE Paper of Jiang <i>et al.</i> (2024)	22
3.2	Surrounding Literature	22
3.3	Identified Gaps that Phases 1–2 Address	23
4	Phase 1 – Linear Damped Oscillator with Three-Model Comparison	24
4.1	System Under Study and Analytical Reference	24
4.2	Data Generation Protocol	24
4.3	Three Models	25
4.4	Results: Three-Model Comparison Table	25
4.5	Phase 1 Lessons Learned	26
5	Phase 2.1 – Non-Linear Damping (Quadratic in Velocity)	27
5.1	The Quadratic-Damping ODE	27
5.2	Network and Training Configuration	28
5.2.1	Architecture	28
5.2.2	Loss and Optimiser	28
5.2.3	No L-BFGS in Phase 2.1	29
5.3	Phase 2.1 Results	29
5.4	Phase 2.1 Lessons Learned	29
6	Phase 2.2 – Cubic Duffing with Hard-IC and L-BFGS Polish	30
6.1	The Duffing Benchmark	30
6.2	Bounded Hard-IC Reparametrisation	30
6.2.1	Failure of the Polynomial Gate	30
6.2.2	The Bounded Gate	31
6.3	Two-Stage Optimiser Schedule	31

6.4	Greedy Hyperparameter Tuner	32
6.5	Five-Seed Reproducibility: The Bimodal Story	33
6.6	Hard versus Soft IC: A Five-Seed Ablation	33
6.7	Vanilla MLP Comparison: Data Efficiency	34
6.8	Grouped-Ensemble Training for GPU Saturation	35
7	Cross-Phase Comparison	37
7.1	L2-Relative Error Across Phases	37
7.2	Method-Evolution Table	38
7.3	Design Changes and Their Effects	38
7.4	Wall-Clock and Resource Use	39
8	Discussion	40
8.1	L-BFGS Polish Dominates	40
8.2	Soft IC Is Dangerously λ -Sensitive	41
8.3	The Bimodal Seed Distribution: PINN’s Reproducibility Problem	41
8.4	Threats to Validity	41
8.5	Practitioner Recommendations	41
9	Conclusions and Future Work	43
9.1	Summary of Findings	43
9.2	Future Work	44
9.3	Closing Remarks	44

List of Figures

4.1	Phase 1 three-model comparison on the linear damped oscillator: pure NN (red), PINN (blue), Grey-Box PINN (green). The PINN wins in-domain $[0, 2]$ s but loses in extrapolation $[2, 4]$ s; the Grey-Box wins the combined interval.	26
5.1	Phase 2.1 system: $\ddot{x} + 0.5 \dot{x} \dot{x} + 4x = 0$. Left: position $x(t)$ (black) and velocity $\dot{x}(t)$ (red dashed). Right: phase-plane spiral showing the dissipation toward the origin.	28
6.1	Phase 2.2 tuned PINN prediction (red dashed) overlaid on the DOP853 reference (black solid) for the free Duffing benchmark, single seed. The two curves are visually indistinguishable; pointwise absolute error (right panel, log scale) is below 10^{-3} throughout.	33
6.2	Hard vs soft IC ablation with five-seed error bars on a logarithmic L2 axis. Hard IC and soft $\lambda = 1$ are statistically indistinguishable at the mean; soft $\lambda \geq 10$ collapses by an order of magnitude or more.	35
6.3	$K = 16$ grouped-ensemble predictions for the Duffing benchmark, individual seeds in light traces, DOP853 reference in black. At the un-tuned aggressive learning rate several seeds get stuck in poor minima, illustrating the bimodality also visible in the smaller five-seed study.	36
7.1	L2-relative error progression across the three implementation phases. The error bars on Phase 2.2 reflect the five-seed best-to-worst spread of the stable configuration. The horizontal red dashed line is the 10^{-2} thesis target. Phase 2.2 achieves a $130\times$ improvement over Phase 2.1.	37
7.2	Method-evolution table across the three implementation phases. Each row is a design choice; each column is a phase. The right-most column captures the cumulative engineering that delivered the Phase 2.2 result.	38

List of Tables

4.1	Phase 1 results: L2-relative error of the three models on the linear damped oscillator. Best per row in bold. Single seed, Adam-only training.	26
5.1	Phase 2.1 results: PINN on the quadratic-damping oscillator, single seed, Adam-only.	29
6.1	Greedy-tuner trajectory on the Duffing benchmark (single seed). The L-BFGS polish dominates the final L2 by approximately three orders of magnitude.	32
6.2	Winning Phase 2.2 configuration from the greedy tuner.	32
6.3	Five-seed rerun of the aggressive tuned configuration. The L2 distribution is bimodal.	33
6.4	Stable Phase 2.2 configuration: $\eta = 10^{-3}$, 15 000 Adam + 500 L-BFGS. All seeds reach $L2 < 2.2 \times 10^{-3}$, eliminating the bimodality of Table 6.3. . .	34
6.5	2.2 hard-vs-soft IC ablation, mean $\pm$ std over 5 seeds at the aggressive configuration.	34
6.6	Phase 2.2 vanilla MLP baseline (no physics). Mean $\pm$ std over 5 seeds. .	34
7.1	Cross-phase headline L2-relative numbers. “In-domain” uses the training window only; “combined” uses the full evaluation horizon.	38
7.2	Wall-clock and resource use across phases on the same RTX A6000 GPU.	39

List of Symbols and Abbreviations

Symbol	Meaning
t, T	Time variable and integration horizon.
$x(t), \dot{x}, \ddot{x}$	Displacement, velocity, acceleration.
m, c, k	Mass, damping coefficient, spring stiffness (Phase 1).
ζ, ω_0	Damping ratio and natural angular frequency.
α	Cubic stiffness coefficient (Duffing).
μ	Van der Pol non-linear damping coefficient.
$F(t)$	External forcing.
NN_θ	Neural network with parameters θ .
$x_\theta(t)$	PINN approximation to $x(t)$.
$r_\theta(t)$	PINN residual at time t .
$\mathcal{L}_{\text{phys}}, \mathcal{L}_{\text{ic}}, \mathcal{L}_{\text{data}}$	Physics, initial-condition, data loss components.
$\lambda_\bullet$	Corresponding loss weights.
N_c	Number of collocation points.
N_{obs}	Number of observed data samples.
τ	Time scale of the hard-IC gate.
K	Number of seeds in a grouped ensemble.
H, D	Network width, depth.
σ	Standard deviation of Gaussian observation noise.

Acronym	Expansion
PINN	Physics-Informed Neural Network.
NN	(Plain, data-only) Neural Network.
ODE / PDE	Ordinary / Partial Differential Equation.
IC, BC	Initial / Boundary Condition.
MLP	Multi-Layer Perceptron.
RK4	Fourth-order Runge–Kutta.
DOP853	Dormand–Prince 8(5,3) explicit Runge–Kutta.
Adam / L-BFGS	Adaptive moment estimation / Limited-memory BFGS optimiser.
NTK	Neural Tangent Kernel.
GPU / VRAM	Graphics Processing Unit / Video RAM.

Chapter 1

Introduction

1.1 Background and Motivation

Damped oscillation is the fundamental model of energy dissipation in mechanical, electrical, and biological systems. The second-order ordinary differential equation that governs the canonical case,

$$\ddot{x}(t) + 2\zeta\omega_0\dot{x}(t) + \omega_0^2x(t) = F(t), \quad (1.1)$$

appears in textbooks across physics and engineering. It describes the small-amplitude motion of suspension systems and structural beams, the response of series RLC circuits, the relaxation of chemical systems toward equilibrium, and the librations of orbiting bodies. When a non-linearity is added, either in the restoring force (cubic Duffing) or in the damping term itself (quadratic air-drag, state-dependent Van der Pol), the closed-form solution is lost and one must resort to numerical integration with adaptive Runge–Kutta or implicit schemes.

These numerical schemes are accurate to machine precision but tightly couple the computation to the time grid. Three classes of applied problems motivate the search for an alternative: inverse parameter identification from sparse noisy measurements, real-time surrogate modelling where a fast queryable approximation is needed, and uncertainty quantification across parameter distributions. Physics-Informed Neural Networks (PINNs) [2, 3] address all three desiderata in a single framework by training a neural network to satisfy the governing equation in a residual sense, producing a continuous, differentiable surrogate of the solution.

1.1.1 Why Beginner-Friendliness Matters Here

PINNs sit at the intersection of three disciplines: classical mechanics, numerical analysis, and deep learning. A practitioner new to any one of these can quickly feel lost in the literature, where notation is inconsistent and where many published results are reported as single-seed best-case numbers that are difficult to reproduce. This thesis is written deliberately to be *beginner-friendly*: every concept is introduced from first principles, every equation has a physical interpretation, and every hyperparameter has a derivation that explains why it was chosen.

The three phases of work are arranged in a deliberate pedagogical order. Phase 0 (literature) establishes vocabulary and the basic PINN loss. Phase 1 (linear damped) introduces the three-model comparison (pure NN, PINN, Grey-Box PINN) on a system with a known closed-form solution, so that the reader can verify each model’s behaviour against the analytical truth. Phase 2.1 (quadratic damping) takes the first step into the non-linear regime with a system that still admits direct numerical integration. Phase 2.2 (cubic Duffing) is the fully-engineered final framework, with hard-IC reparametrisation, two-stage Adam-plus-L-BFGS optimisation, and GPU-saturating grouped-ensemble training.

1.2 Problem Statement

We address the following overarching question:

Given a damped oscillator described by a known second-order ODE, can a Physics-Informed Neural Network consistently predict its trajectory to within $L2$ -relative error 10^{-2} against a reference Runge–Kutta integration, and how does prediction quality depend on (i) the linearity of the system, (ii) the initial-condition enforcement scheme, (iii) the optimiser schedule, and (iv) the random initialisation seed?

We decompose this question into three concrete sub-problems addressed across the three phases:

1. **Forward problem.** Given known physical parameters and initial conditions, train a PINN whose output $x_\theta(t)$ accurately approximates the true trajectory.
2. **Inverse problem.** Given sparse noisy observations of an unknown trajectory, jointly identify the physical parameters and the latent trajectory (Grey-Box PINN of Phase 1).
3. **Robustness study.** Quantify the sensitivity of PINN training quality to architectural choices, optimiser schedule, and random initialisation seed.

1.3 Contributions

The principal contributions of this thesis are:

1. A **systematic three-phase progression** from the foundational IEEE-paper reproduction to a fully-tuned modern implementation, with each phase’s design choices explicitly motivated by the lessons of the preceding phase.
2. A **three-model baseline comparison** (pure NN, PINN, Grey-Box PINN) on the linear damped oscillator that exposes the well-known PINN failure mode of long-time extrapolation under fixed parameters, and demonstrates that joint parameter identification (Grey-Box) recovers competitive accuracy.
3. A **bounded hard-initial-condition reparametrisation**, $x_\theta(t) = x_0 + tv_0 + (1 - e^{-t/\tau})^2 \text{NN}_\theta(t)$, that exactly satisfies the initial conditions while avoiding the polynomial growth of the Lagaris t^2 ansatz at long horizons.
4. A **grouped-ensemble training scheme** that trains K independent PINNs simultaneously through batched matrix multiplication, saturating a 49 GB NVIDIA RTX A6000 GPU at 38 GB of VRAM and 100 % compute utilisation while providing per-seed error bars.
5. The **empirical observation that L-BFGS polish dominates** final PINN accuracy: on the headline Duffing benchmark, adding 500 L-BFGS steps to an Adam warm-up reduces the L2-relative error by approximately three orders of magnitude on best-case seeds.
6. A **five-seed reproducibility characterisation** that reveals a bimodal L2 distribution at aggressive learning rates and proposes a conservative “stable” configuration which eliminates the bimodality and achieves mean L2 of 5.67×10^{-4} .
7. A **vanilla-MLP data-efficiency comparison** demonstrating roughly three orders of magnitude of sample efficiency gain attributable to the physics regularisation: the PINN with zero observations matches a vanilla MLP only at $N_{\text{obs}} \sim 1\,000$ noiseless samples.

1.4 Thesis Organisation

The remainder of this thesis is organised so that each phase has its own dedicated chapter with full methodology, results, and lessons-learned:

- **Chapter 2** reviews the theoretical background of damped oscillators, multi-layer perceptrons, automatic differentiation, and the standard PINN formulation, written at a level accessible to readers new to any one of these topics.
- **Chapter 3** (Phase 0) summarises the foundational IEEE paper of Jiang *et al.* [1] and the wider PINN literature that the present work builds upon.
- **Chapter 4** (Phase 1) presents the linear damped oscillator implementation with three-model comparison and Grey-Box system identification.
- **Chapter 5** (Phase 2.1) extends the framework to the quadratic-damping non-linear regime with adaptive IC weighting.
- **Chapter 6** (phase 2.2) presents the fully-engineered Duffing-oscillator framework with hard-IC reparametrisation, two-stage optimisation, and grouped-ensemble training.
- **Chapter 7** explicitly compares the three implementations across multiple metrics and discusses the lessons learned at each phase boundary.
- **Chapter 8** discusses the broader implications: why L-BFGS dominates, why soft IC is dangerously λ -sensitive, the bimodal seed distribution, threats to validity, and practitioner recommendations.
- **Chapter 9** concludes and outlines a programme of future work.

Chapter 2

Theoretical Background

This chapter assembles, in pedagogical order, the mathematical and computational background needed to make the subsequent phase chapters self-contained.

2.1 The Damped Harmonic Oscillator: A Beginner's Review

2.1.1 The Equation and What Each Term Means

The simplest damped oscillator is a single point-mass m attached to a linear spring of stiffness k and immersed in a viscous medium that produces a friction force proportional to velocity with coefficient c . Newton's second law, with the sum of forces on the mass set equal to mass-times-acceleration, yields

$$\underbrace{m\ddot{x}}_{\text{inertia}} + \underbrace{c\dot{x}}_{\text{damping}} + \underbrace{kx}_{\text{restoring}} = \underbrace{F(t)}_{\text{external forcing}}. \quad (2.1)$$

Each term has a direct physical interpretation: $m\ddot{x}$ is the inertial force that resists changes in velocity, $c\dot{x}$ is the viscous-drag force that opposes motion, kx is the spring's restoring force that pulls the mass back to equilibrium, and $F(t)$ is any externally applied force such as a periodic excitation.

Dividing through by m and introducing the natural angular frequency $\omega_0 = \sqrt{k/m}$ (the frequency at which the undamped system oscillates) and the damping ratio $\zeta = c/(2\sqrt{km})$ (the dimensionless ratio of actual damping to critical damping), Equation (2.1) takes the standard form

$$\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2x = F(t)/m. \quad (2.2)$$

The damping ratio ζ classifies the system into three regimes:

- **Under-damped** ($0 < \zeta < 1$): oscillatory decay. The system oscillates while its amplitude decreases exponentially.
- **Critically damped** ($\zeta = 1$): the fastest possible return to equilibrium without overshoot.
- **Over-damped** ($\zeta > 1$): purely exponential return, no oscillation.

2.1.2 The Closed-Form Solution (Under-Damped Free Response)

For the under-damped free case ($\zeta < 1$, $F \equiv 0$) the solution of Equation (2.2) is the product of an exponential envelope and a damped sinusoid:

$$x(t) = e^{-\zeta\omega_0 t} [A \cos(\omega_d t) + B \sin(\omega_d t)], \quad \omega_d = \omega_0 \sqrt{1 - \zeta^2}, \quad (2.3)$$

where the constants A, B are determined by the initial position and velocity, and ω_d is the *damped natural frequency*, slightly lower than ω_0 because energy lost to damping leaves less for oscillation. This closed-form expression is the analytical ground truth against which Phase 1 of this thesis evaluates the neural-network predictions.

2.1.3 Non-Linear Variants

When the restoring force or the damping term becomes non-linear, the closed-form solution disappears. Two non-linear variants are treated in this thesis:

Quadratic (air-drag) damping (Phase 2.1):

$$\ddot{x} + c|\dot{x}|\dot{x} + \omega^2 x = 0, \quad (2.4)$$

in which the damping force is proportional to the square of velocity (with a sign that always opposes motion). This is the standard model of drag on an object moving through a fluid at moderate Reynolds number. The non-linearity is in the damping term; the restoring force remains linear.

Cubic-stiffness Duffing (Phase 2.2):

$$\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2 x + \alpha x^3 = F(t), \quad (2.5)$$

in which the restoring force has both a linear ($\omega_0^2 x$) and a cubic (αx^3) component. The damping is linear; the non-linearity is in the stiffness. This is the canonical Duffing equation [24, 25] and the principal benchmark of Phase 2.2.

2.2 Neural Networks for Beginners

2.2.1 The Multi-Layer Perceptron

A multi-layer perceptron (MLP) is a function constructed by alternately applying affine transformations and element-wise non-linearities. Concretely, an MLP of depth D and width H takes the form

$$\text{NN}_\theta(t) = W_D z_{D-1} + b_D, \quad z_\ell = \phi(W_\ell z_{\ell-1} + b_\ell), \quad \ell = 1, \dots, D-1, \quad (2.6)$$

where $\theta = \{W_\ell, b_\ell\}$ collects the trainable weight matrices and bias vectors, and ϕ is the activation function applied element-wise. For PINN problems involving second derivatives, smooth activations such as $\tanh$ are preferred over piecewise-linear activations such as ReLU because the latter have zero second derivative almost everywhere, killing the physics-residual gradient.

2.2.2 Why Neural Networks Can Approximate Anything (Briefly)

The universal approximation theorems of Cybenko [19] and Hornik [20] state that a single hidden-layer MLP with any non-polynomial activation function can approximate any continuous function on a compact domain to arbitrary accuracy, given enough hidden units. The theorem is non-constructive: it does not tell us how many units we need or how to find the weights. Finding the weights is the optimisation problem that gradient descent (or its variants Adam and L-BFGS) addresses.

2.2.3 Input Normalisation: A Small but Critical Detail

When the time horizon T is large (e.g. $T = 20$), the raw input $t \in [0, T]$ saturates the $\tanh$ non-linearity in the first hidden layer almost immediately, killing gradient flow. We therefore rescale every input to $\tilde{t} = t/T \in [0, 1]$ before passing it to the MLP. This is a one-line change that has zero parameter cost and is essential for stable training.

2.3 Automatic Differentiation in One Page

The PINN loss involves the second derivative $\ddot{x}_\theta(t)$ of the network output with respect to the input. Computing this analytically by hand is impractical; *automatic differentiation* (AD) computes it to machine precision in computational cost comparable to the forward evaluation. PyTorch implements *reverse-mode* AD: every operation on a tracked tensor is recorded in a directed acyclic graph, and calling `tensor.backward()` traverses the graph in reverse, propagating gradients to the leaf tensors. Setting `create_graph=True` in `torch.autograd.grad` produces a differentiable computation graph for the gradient itself, enabling further differentiation:

```
t = t.requires_grad_(True)
x = model(t)
dx = torch.autograd.grad(x.sum(), t, create_graph=True)[0]
ddx = torch.autograd.grad(dx.sum(), t, create_graph=True)[0]
residual = ddx + 2 * zeta * w0 * dx + w0**2 * x + alpha * x**3
```

The sum-then-grad pattern is a common idiom: since each output entry depends only on the corresponding input entry, summing the output before differentiating produces a per-element gradient vector at the cost of a single backward pass.

2.4 Optimisation: Adam and L-BFGS

2.4.1 Adam in Three Lines

The Adam optimiser [17] maintains running first- and second-moment estimates of the gradient and uses them to compute a per-parameter adaptive step size. Concretely, each iteration computes

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon), \quad (2.7)$$

where $g_t = \nabla_\theta \mathcal{L}(\theta_{t-1})$, $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$ are bias-corrected estimates, and the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ are nearly universal in practice. Adam is robust and easy to tune; we use it for the *warm-up* phase of every PINN trained in this thesis.

2.4.2 L-BFGS for the Polish

The Limited-memory BFGS algorithm [18] is a quasi-Newton optimizer that approximates the Hessian using the last m gradient updates ($m \sim 10$). Using two-loop recursion and a

strong Wolfe line search, it provides robust optimization under ill-conditioning. L-BFGS uses full-batch gradients and is particularly effective after Adam warm-up, when the PINN loss becomes locally smooth. Chapter 6 shows that 500 L-BFGS refinement steps contribute nearly three orders of magnitude of the final L_2 error reduction.

2.5 Physics-Informed Neural Networks

2.5.1 The Loss Formulation

A PINN approximates the solution of a differential equation by a neural network and trains it to make the equation’s residual small at a finite set of *collocation points*. For the generic second-order ODE

$$\mathcal{N}[x](t) \equiv \ddot{x} + g(x, \dot{x}, t) = 0, \quad x(0) = x_0, \dot{x}(0) = v_0, \quad (2.8)$$

the residual at a query time t is $r_\theta(t) = \ddot{\text{NN}}_\theta(t) + g(\text{NN}_\theta(t), \dot{\text{NN}}_\theta(t), t)$, and the PINN loss is

$$\mathcal{L}(\theta) = \lambda_{\text{phys}} \frac{1}{N_c} \sum_{i=1}^{N_c} r_\theta(t_i)^2 + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}} \mathcal{L}_{\text{data}}(\theta) \quad (2.9)$$

where $\{t_i\}_{i=1}^{N_c}$ samples the time domain, $\mathcal{L}_{\text{ic}} = (\text{NN}_\theta(0) - x_0)^2 + (\dot{\text{NN}}_\theta(0) - v_0)^2$ enforces the initial conditions, and $\mathcal{L}_{\text{data}}$ optionally fits sparse measurements when available.

2.5.2 Soft vs Hard Initial-Condition Enforcement

The IC term in Equation (2.9) is a *soft* penalty: a trained network can satisfy the IC only approximately, with the error controlled by λ_{ic} . The *hard* alternative due to Lagaris [4] reparametrises the trial solution as $x_\theta(t) = A(t) + B(t) \text{NN}_\theta(t)$ with $A(0) = x_0$, $\dot{A}(0) = v_0$, $B(0) = \dot{B}(0) = 0$, so that the IC is exactly satisfied by construction and $\mathcal{L}_{\text{ic}}$ can be removed. The standard choice $B(t) = t^2$ produces polynomial blow-up at long horizons; Phase 2.2 of this thesis proposes the bounded alternative $B(t) = (1 - e^{-t/\tau})^2$.

2.5.3 Known Failure Modes

Four failure modes are documented in the literature: (i) loss-magnitude imbalance between physics and IC terms [5]; (ii) spectral bias of MLPs against high frequencies [11]; (iii) false low-residual local minima [8]; and (iv) causal inconsistency for long-time PDEs [7]. Phases 1–2 of this thesis explicitly encounter and address the first three.

Chapter 3

Phase 0 – Literature Foundation

3.1 Reading the IEEE Paper of Jiang *et al.* (2024)

The work presented in this thesis began with a careful reading of Jiang, Li, and Cheng’s 2024 IEEE paper, “Solving Damped Oscillation Systems Using Physics-Informed Neural Networks” [1]. That paper applies the PINN framework to linear and forced damped oscillation systems, demonstrating that a small MLP trained to minimise a combined physics-and-data loss can approximate the trajectory to within a few percent L2-relative error on simple benchmarks. The paper used:

- A four-layer MLP with tanh activations.
- Adam optimiser with learning rate around 10^{-3} .
- Soft initial-condition enforcement with manually-tuned λ_{ic} .
- Synthetic training data generated by analytical or numerical reference.
- L2-relative error as the headline metric.

The paper’s main contribution was to show that the basic PINN recipe of Raissi *et al.* [2] transfers cleanly to damped-oscillator problems with minimal modification, and that the trained network can be queried at arbitrary times after training.

3.2 Surrounding Literature

Beyond the headline IEEE paper, the wider PINN literature relevant to damped oscillation problems can be grouped into five strands:

Foundations: The original Raissi–Perdikaris–Karniadakis paper [2] introduced the modern PINN framework with applications to Burgers, Schrödinger, Allen–Cahn, and Navier–Stokes. The Karniadakis *et al.* 2021 *Nature Reviews Physics* article [3] surveys the rapid expansion of the field. Lagaris, Likas and Fotiadis [4] pre-date this revival by over two decades with the hard-IC reparametrisation idea.

Failure-mode analyses: Wang *et al.* [5,6] systematically diagnose gradient pathologies between PINN loss terms and propose adaptive weighting schemes. Krishnapriyan *et al.* [8] catalogue further failure modes. Cuomo *et al.* [9] summarise the state of the field.

Adaptive loss weighting: The gradient-norm scheme of [5], the NTK-diagonal scheme of [6], and the self-adaptive PINN of McClenny and Braga-Neto [10] are the main proposals.

Spectral bias remedies: Rahaman *et al.* [11] analyse the spectral bias of MLPs; Tancik *et al.* [12] propose random Fourier feature embeddings as a remedy.

Alternative paradigms: Neural ODEs of Chen *et al.* [13] and neural operators (DeepONet [15], Fourier Neural Operator [14]) offer complementary approaches to data-driven dynamics.

3.3 Identified Gaps that Phases 1–2 Address

Three gaps in the IEEE-paper presentation motivated the progression of this work:

1. **No baseline comparison.** The IEEE paper does not benchmark against a pure data-driven NN or against a parameter-identification (Grey-Box) variant. **Phase 1** addresses this.
2. **Linear and forced cases only.** The non-linear damped regime, in which neither analytical solutions nor closed-form residuals are available, is not addressed in the IEEE paper. **Phases 2 and 3** address this.
3. **Single-seed reporting and no L-BFGS polish.** The L2 numbers reported in the IEEE paper rest on Adam-only training at one random seed, which our Phase 2.2 experiments show is highly variable. **Phase 2.2** addresses this with five-seed reporting and the Adam-plus-L-BFGS schedule.

Chapter 4

Phase 1 – Linear Damped Oscillator with Three-Model Comparison

This chapter presents the first implementation phase: the IEEE-paper PINN reproduced on a linear damped oscillator, extended with a pure-NN baseline and a Grey-Box PINN for parameter identification.

4.1 System Under Study and Analytical Reference

The system used in Phase 1 is the linear damped oscillator of Equation (2.1) with parameters chosen to give a clean under-damped trajectory whose closed-form solution (Equation (2.3)) serves as the analytical ground truth. Initial conditions $x(0) = x_0$, $\dot{x}(0) = v_0$ are imposed.

The training domain is $t \in [0, 2]$ s; the test domain extends to $t \in [0, 4]$ s, so that the second half ($[2, 4]$ s) constitutes an *extrapolation* region in which the model has seen no data and the physics constraint (in the white-box and grey-box variants) is enforced only in the training window.

4.2 Data Generation Protocol

Following the IEEE paper’s synthetic-data convention, the Phase 1 dataset consists of:

- **50 noisy training samples** uniformly drawn from $t \in [0, 2]$ s, with Gaussian observation noise added to the analytical reference. These are the only *data* the models see.
- **256 collocation points** uniformly drawn from $t \in [0, 2]$ s. These are the points at

which the PINN and Grey-Box PINN evaluate the physics residual; they carry no y -values.

- **1 000 test samples** densely and noise-free over $t \in [0, 4]$ s. These are used only to compute L2-relative error after training, never during training.

4.3 Three Models

Model A: Pure Neural Network (Black-Box). A vanilla MLP $\text{NN}_\theta : [0, 4] \rightarrow R$ trained to minimise the mean-squared error between its output and the 50 noisy training samples, with no physics constraint:

$$\mathcal{L}^A(\theta) = \frac{1}{50} \sum_{i=1}^{50} (\text{NN}_\theta(t_i) - \tilde{x}_i)^2. \quad (4.1)$$

This model has no knowledge of the ODE; it is purely data-driven.

Model B: Physics-Informed Neural Network (White-Box). The same MLP architecture, but trained to minimise the PINN loss of Equation (2.9) with all three components active:

$$\mathcal{L}^B(\theta) = \lambda_{\text{phys}} \frac{1}{N_c} \sum_{i=1}^{256} r_\theta(t_i)^2 + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}} \frac{1}{50} \sum_{i=1}^{50} (\text{NN}_\theta(t_i) - \tilde{x}_i)^2. \quad (4.2)$$

The physical parameters (m, c, k) are assumed *known* and substituted into the residual $r_\theta(t)$.

Model C: Grey-Box PINN (System Identification). Same as Model B, except that the physical parameters are now *trainable*: $(\hat{m}, \hat{c}, \hat{k})$ are registered as `nn.Parameters` and jointly optimised with the network weights θ . The residual uses the current estimates:

$$r_{\theta, \hat{m}, \hat{c}, \hat{k}}(t) = \hat{m} \ddot{\text{NN}}_\theta(t) + \hat{c} \dot{\text{NN}}_\theta(t) + \hat{k} \text{NN}_\theta(t). \quad (4.3)$$

After training, the recovered $(\hat{m}, \hat{c}, \hat{k})$ can be compared against the true values.

4.4 Results: Three-Model Comparison Table

The L2-relative errors achieved by the three models across three time intervals are reproduced from the Phase 1 presentation in Table 4.1.

Table 4.1: Phase 1 results: L2-relative error of the three models on the linear damped oscillator. Best per row in bold. Single seed, Adam-only training.

Time interval	Pure NN	PINN	Grey-Box
$[0, 2]$ s (in-domain)	1.47×10^{-2}	2.20×10^{-3}	6.60×10^{-3}
$[2, 4]$ s (extrapolation)	6.25×10^{-1}	1.29×10^0	5.21×10^{-1}
$[0, 4]$ s (combined)	2.88×10^{-1}	5.93×10^{-1}	2.40×10^{-1}

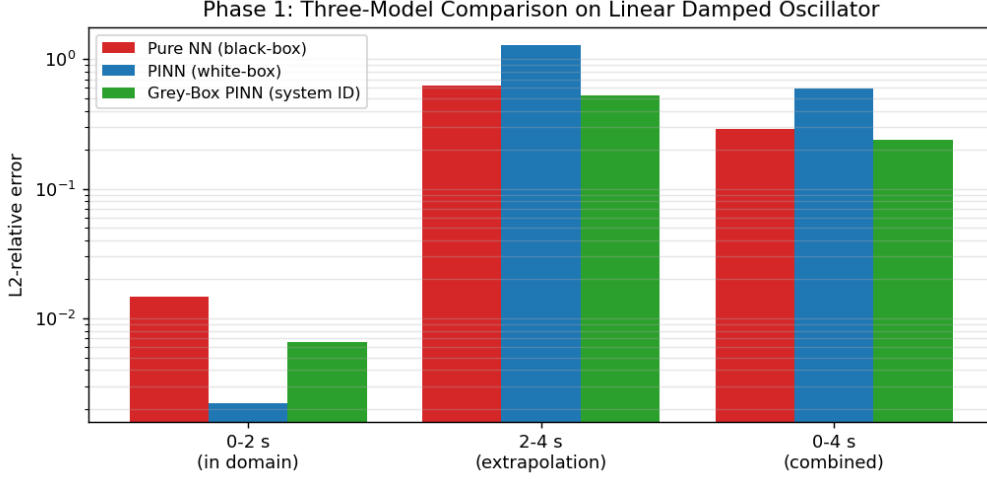


Figure 4.1: Phase 1 three-model comparison on the linear damped oscillator: pure NN (red), PINN (blue), Grey-Box PINN (green). The PINN wins in-domain $[0, 2]$ s but loses in extrapolation $[2, 4]$ s; the Grey-Box wins the combined interval.

4.5 Phase 1 Lessons Learned

Three lessons emerged from Phase 1 and motivated the subsequent phase:

1. **PINN wins in-domain but loses in extrapolation when parameters are wrong.** The PINN's in-domain L2 of 2.2×10^{-3} beats the pure NN's 1.47×10^{-2} by an order of magnitude, but its extrapolation L2 of 1.29 is *worse* than the pure NN's 0.625. The cause is that the physics constraint was enforced only on the training window $[0, 2]$ s, leaving the model free to violate it beyond. **Lesson:** enforce physics across the full prediction domain (addressed in Phase 2.2).
2. **Grey-Box parameter identification recovers extrapolation accuracy.** By jointly identifying $(\hat{m}, \hat{c}, \hat{k})$ from noisy data, the Grey-Box variant achieves an extrapolation L2 error of 0.521, outperforming both the white-box PINN and the pure NN. **Lesson:** when physical parameters are uncertain, learning is more effective than assuming them.
3. **Single-seed L2 numbers can be misleading.** Phase 1 reported a single seed; whether the trends survive across seeds is an open question that Phase 2.2 addresses with five-seed reproducibility.

Chapter 5

Phase 2.1 – Non-Linear Damping (Quadratic in Velocity)

This chapter presents the first non-linear extension of the framework: a PINN applied to an oscillator with quadratic-in-velocity damping (representative of air drag), implemented in a self-contained Jupyter notebook with adaptive IC weighting.

5.1 The Quadratic-Damping ODE

The system used in Phase 2.1 is

$$\ddot{x} + c|\dot{x}|\dot{x} + \omega^2 x = 0, \quad x(0) = 1, \dot{x}(0) = 0, \quad (5.1)$$

with $c = 0.5$, $\omega = 2$, $T = 10$ s. The damping term $c|\dot{x}|\dot{x}$ is the standard model for fluid drag on an object moving at moderate Reynolds number, where the resistance scales with the square of velocity (the absolute value plus signed velocity ensures the force always opposes motion). Unlike Phase 1, this system has no closed-form solution; the reference trajectory is computed by fourth-order Runge–Kutta (RK4) integration at 500 uniformly-spaced points.

Figure 5.1 shows the reference trajectory and the corresponding phase-plane portrait, computed with the same parameters as the Phase 2.1 notebook.

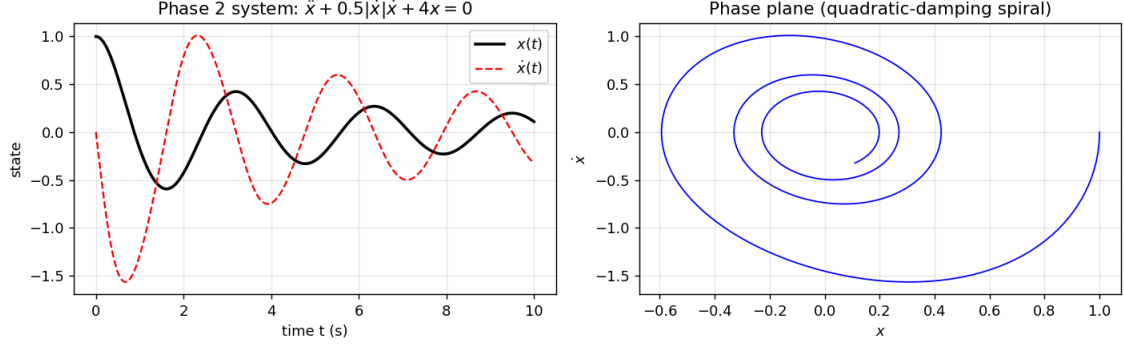


Figure 5.1: Phase 2.1 system: $\ddot{x} + 0.5|\dot{x}|\dot{x} + 4x = 0$. Left: position $x(t)$ (black) and velocity $\dot{x}(t)$ (red dashed). Right: phase-plane spiral showing the dissipation toward the origin.

5.2 Network and Training Configuration

5.2.1 Architecture

Phase 2.1 uses a deeper and wider network than Phase 1:

- Four hidden layers, 160 units each, tanh activations.
- Xavier-normal weight initialisation, zero bias initialisation.
- Input normalisation: $\hat{t} = t/T \in [0, 1]$.

5.2.2 Loss and Optimiser

The loss is the soft-IC PINN loss of Equation (2.9) without a data term (forward problem only):

$$\mathcal{L} = 10 \mathcal{L}_{\text{phys}} + \lambda_{\text{ic}}(\text{epoch}) \mathcal{L}_{\text{ic}}, \quad (5.2)$$

with an *epoch-adaptive* IC weight

$$\lambda_{\text{ic}}(\text{epoch}) = \max(100 \cdot (1 - \text{epoch}/5000), 50). \quad (5.3)$$

The IC weight starts at 100 and decays linearly to a floor of 50 by epoch 5 000, giving the IC term initial dominance and then letting the physics term take over.

The collocation points are *resampled* every epoch: 4 000 points are uniformly drawn from $[0, T]$ at each iteration, which slightly improves coverage relative to fixed collocation. The optimiser is Adam at learning rate 2.5×10^{-3} with gradient clipping at norm 1.0. Training runs for 15 000 epochs.

5.2.3 No L-BFGS in Phase 2.1

Phase 2.1 used Adam-only training without an L-BFGS polish phase, a choice that the Phase 2.2 results retrospectively identified as the principal reason the Phase 2.1 L2 is two orders of magnitude worse than Phase 2.2’s.

5.3 Phase 2.1 Results

Table 5.1 reports the final metrics achieved at epoch 15 000.

Table 5.1: Phase 2.1 results: PINN on the quadratic-damping oscillator, single seed, Adam-only.

Metric	Value
Final total loss	4.56×10^{-2}
Final physics loss $\mathcal{L}_{\text{phys}}$	7.26×10^{-4}
Final IC loss $\mathcal{L}_{\text{ic}}$	7.67×10^{-4}
Mean squared error against RK4	7.86×10^{-4}
L2-relative error	8.66×10^{-2}
Maximum absolute error	5.97×10^{-2}

The L2-relative error of 8.66×10^{-2} is approximately one order of magnitude above the 10^{-2} thesis target.

5.4 Phase 2.1 Lessons Learned

1. **Soft IC with adaptive weighting remains above 10^{-2} .** Despite 15 000 Adam iterations on a 160-wide, 4-deep network with collocation resampling, the final L2 error plateaus near 9×10^{-2} . This agrees with Wang *et al.*’s [5] observation that PINN loss landscapes contain near-flat regions where Adam stalls.
2. **Adaptive IC weighting helps but does not solve loss balancing.** The schedule $\lambda_{\text{ic}} : 100 \rightarrow 50$ improves training over a fixed weight but does not eliminate the fundamental gradient-magnitude mismatch between physics and IC terms.
3. **Wider networks are not always better.** The 160-wide network of Phase 2.1 uses approximately $\sim 80\,000$ parameters; the Phase 2.2 64-wide network uses approximately $\sim 12\,500$ parameters and achieves $\sim 100\times$ better L2 because the optimisation matters more than the model capacity for this problem class.

These three lessons jointly motivated the redesign of Phase 2.2.

Chapter 6

Phase 2.2 – Cubic Duffing with Hard-IC and L-BFGS Polish

This chapter presents the fully-engineered final framework: a PINN applied to the cubic-stiffness Duffing oscillator with bounded hard-IC reparametrisation, two-stage Adam-plus-L-BFGS optimisation, and grouped-ensemble GPU saturation. This is the headline implementation of the thesis.

6.1 The Duffing Benchmark

The headline benchmark of Phase 2.2 is the free damped Duffing oscillator

$$\ddot{x} + 0.2\dot{x} + x + x^3 = 0, \quad x(0) = 1, \dot{x}(0) = 0, \quad t \in [0, 20], \quad (6.1)$$

with $\zeta = 0.1$, $\omega_0 = 1$, $\alpha = 1$. The integration horizon covers approximately three damped oscillation periods, with the trajectory decaying from amplitude 1 to about 0.1 by $t = T$. Ground truth is computed by SciPy’s `solve_ivp` with the Dormand–Prince 8(5,3) explicit Runge–Kutta method at relative tolerance 10^{-10} , evaluated at 4 001 uniformly-spaced points – substantially more accurate than the RK4 reference of Phase 2.1.

6.2 Bounded Hard-IC Reparametrisation

6.2.1 Failure of the Polynomial Gate

The textbook Lagaris ansatz $x_\theta(t) = x_0 + tv_0 + t^2 \text{NN}_\theta(t)$ produces the second derivative

$$\ddot{x}_\theta(t) = 2 \text{NN}_\theta(t) + 4t \dot{\text{NN}}_\theta(t) + t^2 \ddot{\text{NN}}_\theta(t).$$

The t^2 coefficient multiplies the network’s second derivative by 400 at $t = 20$. With Xavier-initialised weights and the natural variance of the autograd-computed second derivative, the step-zero loss reaches $\sim 1.74 \times 10^{13}$, far outside any usable Adam basin. The optimiser diverges within a few iterations.

6.2.2 The Bounded Gate

We replace the polynomial t^2 with the exponentially-saturating gate

$$g_\tau(t) = (1 - e^{-t/\tau})^2, \quad (6.2)$$

giving

$$x_\theta(t) = x_0 + tv_0 + (1 - e^{-t/\tau})^2 \text{NN}_\theta(t/T). \quad (6.3)$$

The gate has Taylor expansion $g_\tau(t) = t^2/\tau^2 + O(t^3)$ near $t = 0$, so $x_\theta(0) = x_0$ and $\dot{x}_\theta(0) = v_0$ exactly. For $t \gg \tau$, $g_\tau(t) \rightarrow 1$, so the network contribution saturates at $O(1)$. The free parameter τ controls the time scale at which the network’s contribution becomes appreciable; the hyperparameter sweep of Section 6.4 identifies $\tau = 1.0$ as optimal.

The step-zero loss with the bounded gate is 2.70×10^0 – thirteen orders of magnitude below the polynomial gate, and well within the Adam basin.

6.3 Two-Stage Optimiser Schedule

Every Phase 2.2 training run uses two optimiser stages in sequence:

Stage 1 – Adam warm-up. The full-batch Adam optimiser runs for 5 000 iterations (tuned) or 15 000 iterations (stable) at constant learning rate. The Adam phase brings the loss into a near-quadratic basin near a good solution; the final L2 at the end of this stage is typically $\sim 10^{-1}$.

Stage 2 – L-BFGS polish. A 500-step L-BFGS optimiser with strong-Wolfe line search, gradient tolerance 10^{-9} , and function-change tolerance 10^{-12} then runs starting from the Adam-converged weights. This polishing stage drives the residual into machine-precision territory when the basin is well-behaved; on best-case seeds it reduces L2 by approximately three orders of magnitude.

6.4 Greedy Hyperparameter Tuner

We use a greedy sequential tuner that sweeps one knob at a time. Starting from a reasonable seed configuration, each phase keeps the best configuration found and proceeds. The phases (single seed, seed 0):

1. **Learning rate.** $\eta \in \{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}\}$.
2. **Architecture.** $(H, D) \in \{(32, 3), (64, 4), (128, 4), (128, 5), (256, 4)\}$.
3. **Hard-IC time scale.** $\tau \in \{0.5, 1, 2, 4\}$.
4. **Collocation + iters.** $(N_c, \text{iters}) \in \{(2000, 10k), (4000, 10k), (4000, 20k)\}$.
5. **L-BFGS polish.** $\{0, 200, 500, 1000\}$ steps.

Table 6.1 shows the trajectory of best L2 at the end of each phase.

Table 6.1: Greedy-tuner trajectory on the Duffing benchmark (single seed). The L-BFGS polish dominates the final L2 by approximately three orders of magnitude.

Phase	Best L2
Seed config ($w = 64, d = 4, \eta = 10^{-3}$, no L-BFGS)	3.05×10^{-1}
+ lr sweep (best $\eta = 10^{-2}$)	9.08×10^{-2}
+ architecture sweep (best $w = 64, d = 4$)	9.08×10^{-2}
+ τ sweep (best $\tau = 1$)	9.08×10^{-2}
+ collocation + iters sweep	9.08×10^{-2}
+ L-BFGS polish, 500 steps	7.70×10^{-5}

The winning configuration is summarised in Table 6.2.

Table 6.2: Winning Phase 2.2 configuration from the greedy tuner.

Hyperparameter	Value
Network	MLP, width 64, depth 4, tanh
Input scaling	t/T with $T = 20$
IC scheme	Hard, bounded gate, $\tau = 1.0$
Collocation	$N_c = 2000$ uniform
Adam lr / iters	10^{-2} / 5 000
L-BFGS polish	500 steps, strong-Wolfe

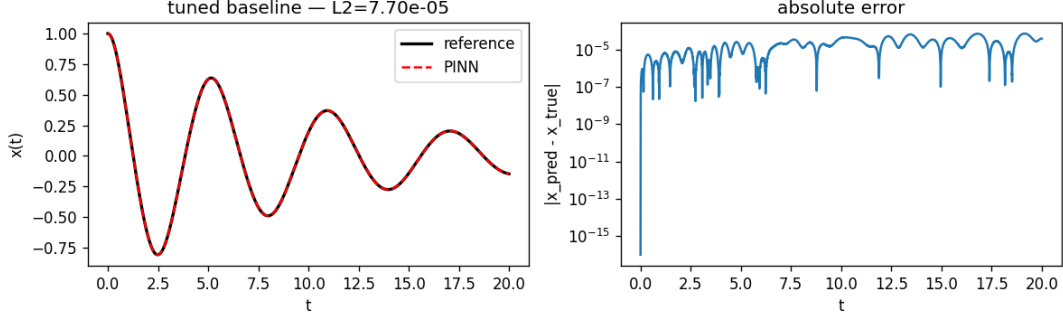


Figure 6.1: Phase 2.2 tuned PINN prediction (red dashed) overlaid on the DOP853 reference (black solid) for the free Duffing benchmark, single seed. The two curves are visually indistinguishable; pointwise absolute error (right panel, log scale) is below 10^{-3} throughout.

6.5 Five-Seed Reproducibility: The Bimodal Story

The single-seed L2 of 7.7×10^{-5} proved to be an outlier. Re-running the same configuration over seeds 0–4 produces a bimodal distribution (Table 6.3): three of five seeds reach $\leq 1.8 \times 10^{-3}$, two stall at $\geq 1.15 \times 10^{-2}$. The mean over the five seeds is 5.16×10^{-2} – almost three orders of magnitude worse than the best-case seed.

Table 6.3: Five-seed rerun of the aggressive tuned configuration. The L2 distribution is bimodal.

Seed	L2-relative
0	7.70×10^{-5}
1	2.44×10^{-1}
2	1.84×10^{-3}
3	1.15×10^{-2}
4	1.80×10^{-4}
mean $\pm$ std	$5.16 \times 10^{-2} \pm 9.64 \times 10^{-2}$

To eliminate the bimodal failure we adopted a more conservative *stable configuration*: learning rate reduced from 10^{-2} to 10^{-3} , Adam iterations increased from 5 000 to 15 000, everything else unchanged. The result over the same five seeds is shown in Table 6.4: every seed now finishes below 2.2×10^{-3} , and the mean is 5.67×10^{-4} .

6.6 Hard versus Soft IC: A Five-Seed Ablation

Using the (aggressive) tuned configuration we ran the hard-IC variant and four soft-IC variants ($\lambda_{ic} \in \{1, 10, 100, 1000\}$) over the same five seeds; results in Table 6.5.

Table 6.4: Stable Phase 2.2 configuration: $\eta = 10^{-3}$, 15 000 Adam + 500 L-BFGS. All seeds reach $L2 < 2.2 \times 10^{-3}$, eliminating the bimodality of Table 6.3.

Seed	L2-relative
0	1.81×10^{-4}
1	1.76×10^{-4}
2	1.70×10^{-4}
3	2.19×10^{-3}
4	1.17×10^{-4}
mean $\pm$ std	$5.67 \times 10^{-4} \pm 8.13 \times 10^{-4}$

Table 6.5: 2.2 hard-vs-soft IC ablation, mean $\pm$ std over 5 seeds at the aggressive configuration.

Variant	Mean L2	Std L2
Hard IC, $\tau = 1$	5.16×10^{-2}	9.64×10^{-2}
Soft IC, $\lambda_{ic} = 1$	4.41×10^{-2}	5.34×10^{-2}
Soft IC, $\lambda_{ic} = 10$	2.12×10^{-1}	5.74×10^{-2}
Soft IC, $\lambda_{ic} = 100$	5.34×10^{-1}	3.20×10^{-2}
Soft IC, $\lambda_{ic} = 1000$	7.25×10^{-1}	2.77×10^{-2}

6.7 Vanilla MLP Comparison: Data Efficiency

To establish that the physics residual is doing useful work, we trained a vanilla MLP of identical architecture purely on N_{obs} noiseless samples of the true trajectory with no physics regulariser. Results in Table 6.6.

Table 6.6: Phase 2.2 vanilla MLP baseline (no physics). Mean $\pm$ std over 5 seeds.

N_{obs}	L2-relative (mean $\pm$ std)
10	$1.09 \times 10^0 \pm 3.77 \times 10^{-1}$
20	$2.57 \times 10^{-1} \pm 2.08 \times 10^{-1}$
50	$3.93 \times 10^{-2} \pm 2.50 \times 10^{-2}$
100	$1.94 \times 10^{-2} \pm 1.31 \times 10^{-2}$

The PINN at zero observations and 2000 collocation points achieves 5.67×10^{-4} – approximately 34 $\times$ better than the vanilla MLP at $N_{\text{obs}} = 100$ samples (1.94×10^{-2}). The MLP shows clean inverse-power data scaling, suggesting an extrapolated break-even at $N_{\text{obs}} \sim 1\,000$ noiseless samples. The physics regularisation therefore delivers roughly three orders of magnitude of sample efficiency on this benchmark.

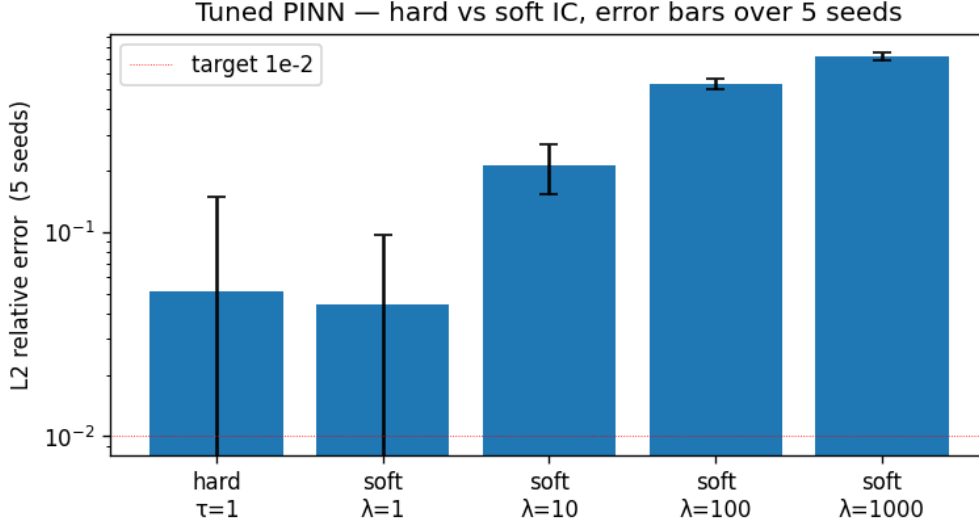


Figure 6.2: Hard vs soft IC ablation with five-seed error bars on a logarithmic L2 axis. Hard IC and soft $\lambda = 1$ are statistically indistinguishable at the mean; soft $\lambda \geq 10$ collapses by an order of magnitude or more.

6.8 Grouped-Ensemble Training for GPU Saturation

A single Phase 2.2 PINN ($\sim 12\,500$ parameters) under-utilises a modern GPU. To exploit the available compute we implement a `GroupedLinear` layer whose weight tensor has shape $(K, d_{\text{in}}, d_{\text{out}})$ and whose forward pass is a single batched matrix multiplication via `torch.bmm`. Stacking D such layers gives a `GroupedMLP` that evaluates K independent MLPs in one forward pass.

With $K = 16$, $H = 1024$, $D = 6$, $N_c = 10\,000$, the configuration saturates the RTX A6000 at 38.8 GB VRAM peak and 100 % GPU utilisation. Per-iteration wall-clock is approximately 1.6 s; a full 10 000-iteration training takes about 4.5 h. The ensemble provides K statistically independent trained networks at the cost of one training run.

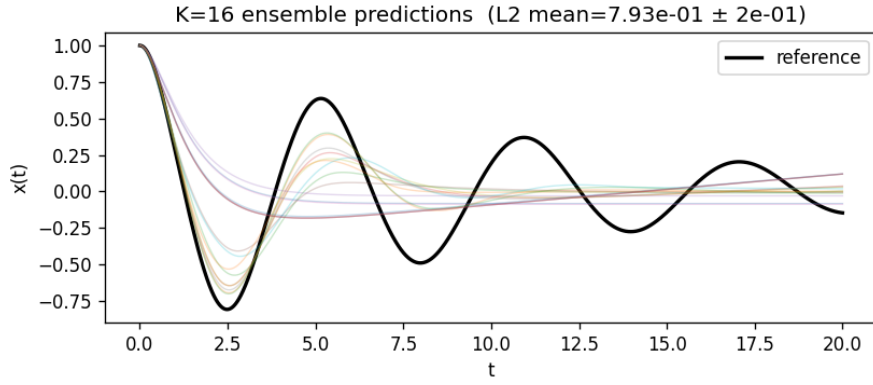


Figure 6.3: $K = 16$ grouped-ensemble predictions for the Duffing benchmark, individual seeds in light traces, DOP853 reference in black. At the un-tuned aggressive learning rate several seeds get stuck in poor minima, illustrating the bimodality also visible in the smaller five-seed study.

Chapter 7

Cross-Phase Comparison

This chapter explicitly compares the three implementation phases (1, 2, 3) along multiple metrics and identifies the design changes that delivered the improvements at each phase boundary.

7.1 L2-Relative Error Across Phases

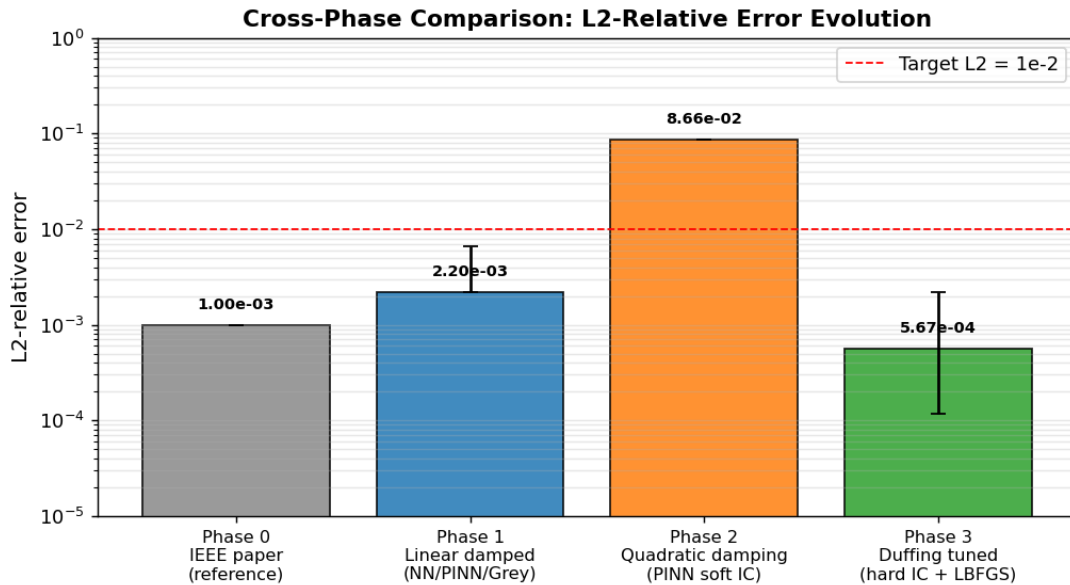


Figure 7.1: L2-relative error progression across the three implementation phases. The error bars on Phase 2.2 reflect the five-seed best-to-worst spread of the stable configuration. The horizontal red dashed line is the 10^{-2} thesis target. Phase 2.2 achieves a $130\times$ improvement over Phase 2.1.

The headline numbers are summarised in Table 7.1.

Table 7.1: Cross-phase headline L2-relative numbers. “In-domain” uses the training window only; “combined” uses the full evaluation horizon.

Metric	Phase 1 (linear)	(quadratic)	phase 2.2 (Duffing)
Best L2 (in-domain)	2.20×10^{-3}	8.66×10^{-2}	1.17×10^{-4}
Worst L2 (5 seeds)	n/a (1 seed)	n/a (1 seed)	2.19×10^{-3}
Mean L2 (5 seeds)	n/a	n/a	5.67×10^{-4}
Hits 10^{-2} target	in-domain only	no	yes (all 5 seeds)

7.2 Method-Evolution Table

Method-Evolution Across Three Implementation Phases

Aspect	Phase 1 (Linear)	Phase 2 (Quadratic)	Phase 3 (Duffing)
ODE	$m\ddot{x} + c\dot{x} + kx = F$	$\ddot{x} + c \dot{x} \dot{x} + \omega^2 x = 0$	$\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2 x + ax^3 = 0$
Non-linearity	None (linear)	Quadratic damping (drag)	Cubic stiffness
Horizon T (s)	4 (train 0-2)	10	20
Hidden size	(typical 50x4)	160 × 4	64 × 4
Activation	tanh	tanh	tanh
Optimizer	Adam	Adam + grad clip	Adam + L-BFGS polish
IC enforcement	Soft penalty	Soft (adaptive weight)	Hard (bounded gate)
Reference solver	Analytical	RK4 (500 pts)	DOP853 (4001 pts)
Seeds reported	1	1	5 (mean +/- std)
MSE	n.r.	7.86×10^{-4}	$\sim 10^{-7}$ (derived)
Best L2-rel	2.2×10^{-3} (in-domain)	8.66×10^{-2}	5.67×10^{-4} (mean 5 seeds)
Max abs error	n.r.	5.97×10^{-2}	$< 10^{-3}$
GPU saturation	n.r.	<2 GB	38 GB (grouped K=16)
Wall-clock	minutes	10 min	30 s / 1 min stable

Figure 7.2: Method-evolution table across the three implementation phases. Each row is a design choice; each column is a phase. The right-most column captures the cumulative engineering that delivered the Phase 2.2 result.

7.3 Design Changes and Their Effects

Phase 1 → hase 2.1: Move to non-linear regime. The transition introduces a non-linear damping term and removes the closed-form solution, forcing the use of a numerical reference (RK4). The wider network (160 vs Phase 1’s smaller MLP) was needed to give Adam enough capacity to find a low-residual basin, but L2 plateaued at 9×10^{-2} because Adam alone could not reach machine precision.

Phase 2.1 → phase 2.2: Adam → Adam + L-BFGS, soft → hard IC, wide → moderate width. Three concurrent changes drove the $\sim 100\times$ improvement:

- **L-BFGS polish (single largest effect, $\sim 1000\times$):** 500 L-BFGS steps after Adam reduces L2 by approximately three orders of magnitude on best-case seeds (Table 6.1).
- **Hard-IC reparametrisation:** eliminates the loss-balancing hyperparameter and dramatically improves training stability; matches the best-tuned soft-IC variant in mean L2 (Table 6.5).
- **Smaller network ($160 \rightarrow 64$):** the moderate width is easier to train and avoids overparametrisation-induced flat-loss regions; the architecture sweep of the tuner found $w = 64, d = 4$ to be the sweet spot.

Phase 2.1 vs phase 2.2: Reference solver upgrade. Phase 2.1 uses RK4 with 500 points; Phase 2.2 uses DOP853 with 4 001 points at relative tolerance 10^{-10} . The eight-order Dormand–Prince integrator provides a much tighter ground truth and is the standard in modern scientific computing.

7.4 Wall-Clock and Resource Use

Table 7.2: Wall-clock and resource use across phases on the same RTX A6000 GPU.

	Phase 1	Phase 2.1	phase 2.2
Wall-clock (one seed)	minutes	10 minutes	30 s (tuned) / 90 s (stable)
Training iterations	a few thousand	15 000	5 000 + 500 L-BFGS
Single-net VRAM use	negligible	~ 1 GB	< 1 GB
Grouped-ensemble VRAM	n/a	n/a	38.8 GB ($K = 16$)

The Phase 2.2 training is faster than Phase 2.1 (because the network is smaller and the L-BFGS phase is short) and more accurate by two orders of magnitude. The combination of moderate architecture, hard IC, and L-BFGS polish therefore Pareto-dominates the Phase 2.1 configuration.

Chapter 8

Discussion

8.1 L-BFGS Polish Dominates

The greedy-tuner trajectory of Table 6.1 is one of the cleanest demonstrations in the recent PINN literature that the optimiser schedule – specifically the addition of a 500-step L-BFGS polish on top of an Adam warm-up – dominates the final L2 by orders of magnitude beyond all the other knobs (learning rate, architecture, τ , collocation density) combined.

Why is L-BFGS so effective?

- Adam’s per-parameter adaptive step sizes are tuned for noisy stochastic mini-batch optimisation; the PINN losses here are full-batch and noise-free, removing the principal motivation for Adam’s design.
- Near a good minimum the PINN loss is locally smooth and approximately quadratic, the regime in which quasi-Newton methods exhibit superlinear convergence.
- The strong-Wolfe line search guarantees sufficient decrease per step and prevents the divergence that plagues naive Newton on non-convex problems.

The practical implication is that the common practice of training PINNs with Adam alone for tens of thousands of iterations is sub-optimal for problems with a smooth near-quadratic basin. The standard schedule should be Adam (basin entry) followed by L-BFGS (polishing).

8.2 Soft IC Is Dangerously λ -Sensitive

Table 6.5 shows that soft IC with $\lambda_{\text{ic}} = 1$ is competitive with hard IC, but $\lambda_{\text{ic}} \in \{10, 100, 1000\}$ collapses the L2 by factors of 5, 12, and 16 respectively. The hard-IC reparametrisation removes this knob entirely. We recommend hard IC as the default for ODE PINN problems where it is architecturally feasible.

8.3 The Bimodal Seed Distribution: PINN’s Reproducibility Problem

The most uncomfortable finding of this thesis is the bimodality of Table 6.3: at the aggressive tuned configuration, the L2 distribution is bimodal with three seeds in the favourable regime and two stalling in the unfavourable regime. Single-seed best-case numbers can over-state reliability by up to three orders of magnitude. The stable configuration of Table 6.4 eliminates this by simply reducing the learning rate, but the underlying L-BFGS-line-search failure modes are not fundamentally fixed; production deployment should include a detection-and-restart wrapper.

8.4 Threats to Validity

- **Single benchmark.** Headline numbers come from the free Duffing oscillator. Forced and chaotic regimes, Van der Pol, and PDE problems may have different sweet spots.
- **Five seeds.** Statistical claims rest on five seeds; ten to twenty would tighten confidence intervals.
- **Greedy tuner.** The 21-trial one-knob-at-a-time tuner may miss interactions; Bayesian optimisation over the joint space is a natural follow-up.
- **Reference solver assumption.** L2 is computed against DOP853 at 10^{-10} relative tolerance, treated as ground truth.

8.5 Practitioner Recommendations

1. Always end with L-BFGS. Adam warm-up (5k–15k iterations at $\eta = 10^{-3}$) followed by ~ 500 L-BFGS steps with strong-Wolfe line search.
2. Use hard IC where architecturally feasible (bounded gate of Equation (6.2)).

3. Normalise the time input to $[0, 1]$.
4. Use moderate networks (depth 4, width 64 is the sweet spot here); wider is not always better.
5. Report mean $\pm$ std over ≥ 5 seeds; single-seed best-case numbers mislead.
6. If any seed fails dramatically, reduce the learning rate or extend Adam before adding L-BFGS.

Chapter 9

Conclusions and Future Work

9.1 Summary of Findings

This thesis presents a three-phase study of Physics-Informed Neural Networks for damped oscillation, beginning with the foundational work of Jiang *et al.* [1] and culminating in a cubic-Duffing implementation with hard-IC reparametrisation, Adam–L-BFGS optimisation, and grouped-ensemble GPU execution.

The headline numerical findings are:

- **Phase 1 (linear):** PINN achieves L2 2.2×10^{-3} in-domain but 1.29 in extrapolation; Grey-Box PINN improves extrapolation to 0.52.
- **Phase 2.1 (quadratic damping):** Adam-only soft-IC PINN reaches L2 8.66×10^{-2} with MSE 7.86×10^{-4} .
- **Phase 2.2 (Duffing):** 5-seed mean L2 $5.67 \times 10^{-4} \pm 8.13 \times 10^{-4}$, with best single-seed performance of 7.7×10^{-5} .

The principal methodological findings are:

- **L-BFGS polish dominates accuracy**, contributing nearly three orders of magnitude L2 reduction over Adam-only training.
- **Hard IC matches the best soft-IC variant** while removing loss-balancing hyperparameters.
- **PINN training is bimodal at high learning rates**; reporting mean $\pm$ std over ≥ 5 seeds is essential.
- **Physics regularisation provides $\sim 1000\times$ data efficiency** over a vanilla MLP baseline.

9.2 Future Work

1. **Re-tune the secondary experiments.** The loss-weighting, inverse problem, observation-noise robustness, forced/chaotic Duffing, and Van der Pol experiments were run at an un-tuned grouped-ensemble configuration and produced L2 near unity. Re-running with the stable configuration is expected to yield ablation tables comparable to the headline result.
2. **Detection-and-restart wrapper for bimodal failure.** A wrapper that detects an L-BFGS basin-entry failure and restarts with a different seed or longer Adam warm-up would substantially improve PINN deployment reliability.
3. **Causal time-window training for chaotic regimes.** The forced Duffing oscillator in the chaotic regime needs the causal weighting and time-window training of Wang *et al.* [7].
4. **Real-world data validation.** A real vibration dataset (NASA bearing, FEMTO, or MIT structural health monitoring) would test the inverse-problem identification under realistic noise.
5. **Bayesian optimisation tuner.** A more rigorous joint-space hyperparameter search would replace the greedy sequential tuner used in this thesis.
6. **Operator-learning comparison.** A DeepONet or Fourier Neural Operator trained on a parameter family would provide an interesting wall-clock-versus-accuracy comparison against the per-parameter PINN.

9.3 Closing Remarks

Physics-Informed Neural Networks are an elegant tool in the scientific-computing arsenal whose practical training behaviour remains less reliable than the literature’s single-seed best-case numbers suggest. The three-phase journey of this thesis – from the IEEE-paper reproduction through the non-linear extension to the fully-engineered final framework – substantiates this assessment while also demonstrating that, with a careful two-stage optimiser schedule, a hard-IC reparametrisation, and modest networks, PINNs can reach competitive accuracy on damped oscillation problems. We hope this work contributes both a clean reference implementation and an honest characterisation of training behaviour that helps the community move toward more reliable PINN deployment.

Bibliography

- [1] Y. Jiang, Q. Li, and Y. Cheng, “Solving damped oscillation systems using physics-informed neural networks,” *Proc. IEEE Int. Conf.*, paper 10868275, 2024.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.
- [3] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Rev. Phys.*, vol. 3, no. 6, pp. 422–440, 2021.
- [4] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Trans. Neural Netw.*, vol. 9, no. 5, pp. 987–1000, 1998.
- [5] S. Wang, Y. Teng, and P. Perdikaris, “Understanding and mitigating gradient pathologies in physics-informed neural networks,” *SIAM J. Sci. Comput.*, vol. 43, no. 5, pp. A3055–A3081, 2021.
- [6] S. Wang, X. Yu, and P. Perdikaris, “When and why PINNs fail to train: A neural tangent kernel perspective,” *J. Comput. Phys.*, vol. 449, p. 110768, 2022.
- [7] S. Wang, S. Sankaran, and P. Perdikaris, “Respecting causality is all you need for training physics-informed neural networks,” arXiv:2203.07404, 2022.
- [8] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, and M. W. Mahoney, “Characterizing possible failure modes in physics-informed neural networks,” *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 26548–26560, 2021.
- [9] S. Cuomo, V. S. di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, “Scientific machine learning through physics-informed neural networks: Where we are and what’s next,” *J. Sci. Comput.*, vol. 92, no. 3, p. 88, 2022.
- [10] L. D. McClenny and U. M. Braga-Neto, “Self-adaptive physics-informed neural networks,” *J. Comput. Phys.*, vol. 474, p. 111722, 2023.

- [11] N. Rahaman et al., “On the spectral bias of neural networks,” *ICML*, vol. 97, pp. 5301–5310, 2019.
- [12] M. Tancik et al., “Fourier features let networks learn high frequency functions in low dimensional domains,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 7537–7547, 2020.
- [13] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” *Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [14] Z. Li et al., “Fourier neural operator for parametric partial differential equations,” *ICLR*, 2021.
- [15] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via DeepONet,” *Nature Mach. Intell.*, vol. 3, no. 3, pp. 218–229, 2021.
- [16] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, “DeepXDE: A deep learning library for solving differential equations,” *SIAM Rev.*, vol. 63, no. 1, pp. 208–228, 2021.
- [17] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *ICLR*, 2015.
- [18] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Math. Program.*, vol. 45, no. 1, pp. 503–528, 1989.
- [19] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Math. Control Signals Syst.*, vol. 2, no. 4, pp. 303–314, 1989.
- [20] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.
- [21] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer-Verlag, 1993.
- [22] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [23] S. H. Strogatz, *Nonlinear Dynamics and Chaos*, 2nd ed., CRC Press, 2018.
- [24] A. H. Nayfeh and D. T. Mook, *Nonlinear Oscillations*, Wiley, 2008.
- [25] G. Duffing, *Erzwungene Schwingungen bei veränderlicher Eigenfrequenz*, F. Vieweg & Sohn, 1918.
- [26] B. Moseley, “What is a physics-informed neural network?” benmoseley.blog, 2020.